

Documentation: NASDAQ ITCH 5.0 Data Parser And Order Book Generator

Overview:

This software is a low latency C++ system designed to parse binary NASDAQ 5.0 ITCH data, construct the Limit Order Book for a specific target asset (here, VOD) and write it to a CSV file. It utilises a 2 thread Producer-Consumer architecture as follows-

1. The Producer Thread (Main thread): Memory-maps the file and parses the messages, then pushes event structs into a lock free (SPSC Ring Buffer) queue according to the message type.
2. The Consumer Thread (orderBookConsumerThread): Does most of the heavy lifting, which includes maintaining active order states and sorted price levels, computing best bid/ask prices and writing them to the csv file.

Core Technical Concepts:

1. Memory-Mapping (Using mmap): It allows the application to read file data directly via memory pointers, eliminating overhead of standard read() system calls.
2. SPSC Ring Buffer (Single Producer Single Consumer): It is a data structure where one producer thread writes data and one consumer thread reads data in a circular queue. It avoids mutex(mutual exclusion, when 2 threads try to modify the same data hence the OS has to intervene) by leveraging hardware level atomics.
3. Endianess: NASDAQ ITCH transmits data in the Big Endian format, which needs to be converted into Little Endian (used by computers) via bit manipulation.
4. Struct Packing: It should be ensured that the message type structs do not contain any natural memory alignment packing by the OS as they need to be aligned with the raw bytes of the data stream.

Flow of Code:

1. #include Statements: For various data types and other low level operations used in the code.
2. Struct Definitions: Define 6 structs for each Order type with no padding (using #pragma pack), and a general OrderEvent and LocalOrder struct used in the Consumer thread.
3. SPSC Ring Buffer Definition: Define the SPSC Ring Buffer with alignas(64) to make sure that the head and tail are placed on separate cache lines.
4. Swapping functions: To swap Big to Little Endian, defined as inline to minimise function call overhead.
5. Function writeOrderBook: Takes maps of bids and asks as arguments (which are already sorted), then writes Top 3 best bids/asks and other information to the CSV file.

6. `orderBookConsumerThread`: It constructs the “bids” and “asks” map at each tick by reading the message type and appropriately modifying active orders. Also calls the `writeOrderBook` function in the end.
7. `Main` function: Parses the raw binary file first, then looks for required stock messages by comparing the Stock Locate of incoming messages. It then pushes these messages to the Ring Buffer with the appropriate message type and data.

Key Optimizations:

1. **Zero Copy File Reading**: Standard I/O methods copy data multiple times between kernel space and user space buffers, introducing significant latency. Instead used `mmap()` to map the entire file into virtual memory first.
2. **Fast Path Filtering**: Running a string comparison like `memcmp()` for ticker symbol (“VOD”) would create massive overhead. Instead used the Stock Locate value to check for any matches after the first time.
3. **Lock Free Thread Communication**: Thread synchronization using mutexes introduces OS context switching overhead. The SPSC Ring Buffer allows fast and secure lock free communication using atomic variables for head and tail.
4. **Order Book State Maintenance**: Using `unordered_map` for looking up active orders via Order Reference ID provides $O(1)$ lookup time complexity, but using maps for bids/asks allows us to keep them sorted and determine best bid/ask quickly.
5. **CSV Serialization**: Writing strings using `fprintf` too many times is heavy and expensive. Instead a character buffer (`setvbuf`) is maintained and is flushed in large blocks.

Conclusion:

This project demonstrates the application of low-level C++ system design principles to achieve low latency in parsing raw binary exchange feeds and processing that data into structured insights. In quantitative finance and algorithmic trading, performance is paramount and a software even a few microseconds slower represents lost market opportunities.

From a systems architecture standpoint, this codebase provides a scalable template that can be extended further down the hardware stack. Future iterations could integrate kernel-bypass network frameworks, or shift state maintenance containers into lock-free memory pools to completely eliminate dynamic heap allocations during runtime.